

LINUX BASIC I/O

The Secret Language of Files

THE "EVERYTHING IS A FILE" PHILOSOPHY

In Unix and Linux, this is more than just a phrase—it's a core design principle.

By treating diverse resources as files, the OS provides a **unified interface** for communication.

Whether you're writing to a document or sending data to a printer, you use the same set of system calls.



Normal Files

Documents, source code, images, and media.



Device Files

Hard drives (/dev/sda), keyboards, and terminals.



Special Files

Named pipes, FIFOs, and IPC objects for communication.

"The same tools for everything."

THE RESTAURANT ANALOGY



The Customer

You want a specific dish (data/action)
but can't enter the kitchen.



The Waiter

The messenger who takes your order
and brings back the food.



The Kitchen

Where the magic happens. Only
authorized staff (Kernel) can cook.

Technical Mapping:

User Application

System Call (API)

Kernel / Hardware

THE UNIVERSAL TOOLBOX

`open()`

The entry point. Obtains a "ticket" (File Descriptor) to access a file or device. Defines your access mode.

`read()`

Pulls data from the file/device into your program's memory buffer. Moves the internal cursor forward.

`write()`

Pushes data from your program's buffer out to the file or hardware. Essential for saving or sending data.

`close()`

The cleanup step. Releases the File Descriptor and tells the kernel you're finished with the resource.

CRITICAL RULE: You **MUST** call `open()` before you can `read()` or `write()`!

OPEN() - GETTING YOUR ENTRY TICKET

The Syntax

```
int fd = open(path, flags, mode);
```

path: The name or location of the file.

flags: How you intend to use the file.

mode: Permissions (only used when creating).

*Permissions are expressed in **Octal** (e.g., 0666) to define Read/Write/Execute access for User, Group, and Others.*

Flag	Description
O_RDONLY	Open for reading only.
O_WRONLY	Open for writing only.
O_RDWR	Open for both reading and writing.
O_CREAT	Create the file if it doesn't exist.
O_TRUNC	Wipe existing content if file exists.
O_APPEND	Write data at the end of the file.

THE FILE DESCRIPTOR (THE "CLAIM CHECK")

What is an FD?

When you `open()` a file, the kernel doesn't give you the file itself. It gives you a non-negative integer.

Think of it as a **Claim Check** or a coat-check ticket. You present this number to the kernel whenever you want to read or write.

This number is an index into the **File Descriptor Table** stored in the Process Control Block (PCB).

FD	Name	Default Device
0	stdin	Keyboard
1	stdout	Screen / Terminal
2	stderr	Screen (for Errors)
3+	User Files	Disks, Sockets, etc.

i Your first opened file usually gets FD 3 because 0, 1, and 2 are inherited from the parent process.

INODE: THE FILE'S PASSPORT

The **Inode** (Index Node) is the source of truth for every file on your disk.

It doesn't store the actual data, but it stores everything *about* the data.

***Fun Fact:** The filename isn't stored in the Inode! It's stored in the directory file, which points to the Inode ID.*

>_ Try: `ls -li`

FILE METADATA

Inode ID:	#842910
File Size:	4096 Bytes
Owner UID:	1000 (ubuntu)
Permissions:	0644 (rw-r--r--)
Timestamps:	Created, Modified, Accessed
Data Blocks:	Pointers to Disk Sectors

FILE OBJECT: THE "OPEN BOOK" SESSION

📖 Runtime Session

While the Inode is the file's "Passport," the **File Object** represents a specific open session.

- 🔖 **f_cursor:** Tracks where you are currently reading or writing (the bookmark).
- 🛡️ **f_modes:** Stores the access mode (Read-Only, Write-Only, etc.) for this session.
- 🚩 **f_flags:** Status flags like O_APPEND or O_NONBLOCK.

Key Distinction:

Feature	Inode Object	File Object
Location	Persistent (Disk)	Volatile (RAM)
Existence	Always exists	Only when open
Count	One per file	One per "open()"
Focus	Metadata/Identity	Current State

"If two people read the same book, they have one book (Inode) but two different bookmarks (File Objects)."

ANATOMY OF A FILE LIST (LS -L)

```
-rwxr-xr-x 1 4096 Jun 10 22:30 file.txt
```

Permissions Type & Access	Links Hard Links	Owner User & Group	Size In Bytes	Timestamp Last Modified	Name Filename
------------------------------	---------------------	-----------------------	------------------	----------------------------	------------------

Where does this data come from? **The Inode Object!**

When you run **ls -l**, the command internally uses the **stat()** system call to fetch this metadata from the disk's Inode table and present it to you.

READING: PULLING DATA IN

```
ssize_t ret = read(fd, buffer, count);
```



1. Check FD



2. Get Cursor



3. Fetch Blocks



4. Copy to RAM

The Cursor Rule

The kernel reads from the current **f_cursor** position in the File Object and moves it forward by the number of bytes read.

The Return Value

Returns the actual number of bytes read. If it returns **0**, you've reached the End of File (EOF). **-1** indicates an error.

WRITING: PUSHING DATA OUT

The write() System Call

```
ssize_t ret = write(fd, buf, count);
```

- ✓ **fd**: The File Descriptor (ticket).
- ✓ **buf**: Pointer to data in your program.
- ✓ **count**: Number of bytes to write.

Return Value:

Success: Number of bytes written.
Failure: **-1** (check errno).

O_APPEND

Data is added to the **end** of the file. The cursor is moved to the end before every write.

O_TRUNC

If the file exists and is opened for writing, its length is truncated to **zero** (wiped clean).

O_CREAT

If the file doesn't exist, the kernel will **create** it using the provided permissions.

"Writing is the reverse of reading: data flows from your user-space buffer to the kernel's disk blocks."

SUMMARY: THE JOURNEY OF A SYSTEM CALL

01 `open(filename)` Application requests access to a resource.

02 **Kernel Validation** Kernel finds Inode, creates File Object, and returns an FD.

03 `read()` / `write(fd)` Application uses the FD "ticket" for data transfer.

04 **Driver Interaction** Kernel coordinates with hardware drivers to move bits.

05 `close(fd)` Session ends, resources are released, and the ticket is voided.

Everything is a file, and now you speak the language.